**Changelog**
(12 October 2026 - v1.1 – updated checkpoints; evaluation clarification)
(11 October 2026 - Assignment released)

# Project – End-to-End MLOps Pipeline for Flight Delay Severity Prediction

This project aims to provide students with practical experience in designing, implementing, and managing a complete Machine Learning Operations (MLOps) pipeline. Through a realistic problem, students will apply best practices in automation, reproducibility, deployment, and monitoring to deliver a fully functional predictive system.

The assignment emphasizes the engineering principles such as versioning, testing, continuous integration, and production monitoring that ensure reliability and scalability in real-world machine learning environments.

## 1. Project Overview

The project involves building an end-to-end MLOps workflow to predict the severity of flight arrival delays using historical data from the US Bureau of Transportation Statistics (BTS).

Each team will develop and maintain its own repository containing the complete workflow, while a centralized server will act as the deployment and evaluation environment, simulating a client production system.

The prediction task consists of assigning each flight to one of three arrival delay severity categories:

| Category | Definition |
|---|---|
| On time | Arrival delay below 15 minutes |
| Moderate delay | Arrival delay of at least 15 but less than 60 minutes |
| Severe delay | Arrival delay of at least 60 minutes |

To mirror real-world operational challenges, data will be released in sequential rounds representing future time periods. These rounds will simulate changes in flight characteristics and data distributions. Evaluation will also include controlled scenarios designed to assess inference robustness and the ability to detect performance degradation affecting specific airlines or airports.

Ground-truth labels will become available after predictions have been generated, allowing students to implement and demonstrate monitoring under delayed-label conditions.

## 2. Dataset Description

The dataset consists of historical domestic flight records from the **US Bureau of Transportation Statistics (BTS), Airline On-Time Performance Data**.

Dataset: https://www.transtats.bts.gov/DL_SelectFields.aspx?gnoyr_VQ=FGJ

Each record represents an individual flight and contains a range of attributes related to flight identification, airline and airport information, scheduled and actual flight operations, and delay outcomes.

The predictive target is the **arrival delay severity category**, derived from the actual arrival delay.

Students must use flight records from **2022 and 2023** to develop, train, validate, and evaluate their models. They are responsible for exploring the dataset, identifying relevant features, and implementing appropriate preprocessing and feature engineering. Only information available before the scheduled departure of a flight may be used for prediction.

Flight records from subsequent years will be reserved for centralized evaluation of the deployed models. Only flights that were neither cancelled nor diverted and have a valid recorded arrival delay will be considered for model development and centralized evaluation.

## 3. Project Requirements

Students must implement a complete MLOps pipeline emphasizing automation, reproducibility, and continuous integration. The project is divided into three main components:

**Data and Model Management**

- Implement reproducible data preprocessing pipelines.
- Track experiments and models using MLflow or equivalent.
- Use Git and GitHub for source control and collaboration.
- (Bonus) Integrate DVC for data versioning and provenance tracking.

**B. Automation and Deployment**

- Implement automated testing and model training workflows using GitHub Actions.
- Package the inference service as a Docker container.
- Automate the deployment of the inference service and its required model artifacts through CI/CD pipelines.
- Implement an inference API (e.g., using FastAPI) exposing a /classify endpoint that supports batch inference.
- The endpoint must accept a JSON object containing an instances array of flight records.

- During centralized evaluation, the endpoint will receive raw flight attributes available before scheduled departure, using the **same feature names as in the original BTS dataset**.
- The endpoint must return a classification array containing one predicted delay severity category per input record, preserving the original order.
- Valid prediction categories are on_time, moderate_delay, and severe_delay.

The expected input format is as follows:

```
{
  "instances": [
    {
      "FL_DATE": "2025-05-15",
      …
    },
    {
      "FL_DATE": "2025-05-16",
      …
    }
  ]
}
```

The endpoint should return data in the following format:

```
{

  "classifications": [
    "moderate_delay",
    "on_time"
  ]
}
```

The system must implement appropriate input validation, returning meaningful error responses for invalid or malformed requests, and handle previously unseen categorical values without unexpected failures.

Your system will undergo multiple evaluation rounds once the inference endpoint is available. A registration mechanism will be provided for submitting your endpoint for centralized testing. Each round will introduce new flight data from future months or years, simulating real-world data drift caused by factors such as seasonal variations, changes in flight patterns, or changes in airline and airport coverage. Following each round, the corresponding data and ground-truth labels will be released to your team **after a delay**, simulating realistic label availability and enabling the implementation and demonstration of monitoring mechanisms, drift detection strategies, and optional continuous training procedures.

**C. Monitoring and Drift Detection**

- Monitor overall predictive performance using appropriate evaluation metrics.
- Monitor predictive performance across relevant subgroups, particularly operating airlines and origin airports.
- Detect changes in input feature distributions and predictive performance over time.

- Identify performance degradation affecting specific subgroups, even when overall performance remains relatively stable.
- Define and demonstrate appropriate alerting strategies.
- (Bonus) Implement automated retraining triggered by detected drift or performance degradation.

## 4. Technologies and Architecture

Students may use any programming language of their choice, although Python is strongly recommended due to its extensive ecosystem for data science and MLOps development.

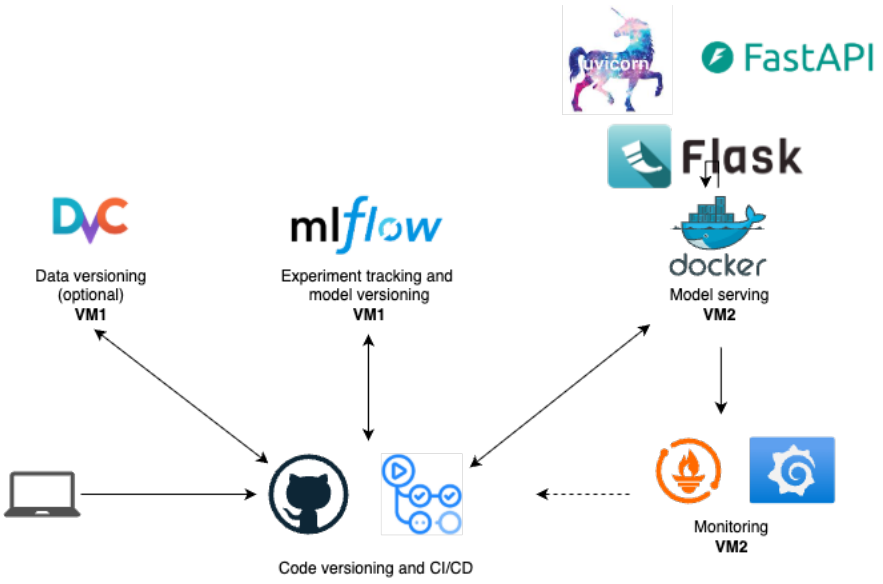
Code versioning must be managed through GitHub, with GitHub Actions used to implement CI/CD workflows. Teams are expected to use MLflow (or equivalent) for experiment tracking and model versioning, while Prometheus and Grafana are recommended for system monitoring and performance visualization.

In addition to their local development environments, each team will require (at least) two virtual machines (VMs): one dedicated to hosting the MLflow (or equivalent) server (and optionally supporting DVC for data versioning) and another serving as the deployment environment, hosting the production model and monitoring tools.

VMs can be requested through the DEI Helpdesk ( https://my.dei.uc.pt/login ). **Teams are expected to adopt computationally efficient approaches and models, simulating the need to minimize resource requirements.**

The computational resources provided by GitHub Actions' free hosted runners may be insufficient for all training and deployment workflows. Therefore, teams are encouraged to configure self-hosted GitHub Actions runners on their VMs, particularly since these machines operate within DEI's network and will not be directly accessible from outside.

A simplified diagram illustrating the overall architecture and its key components is provided below.




## 5. Milestones

| Checkpoint | Date | Deliverable |
|---|---|---|
| Checkpoint | 10 November | Baseline model, code and experiment versioning (Git, MLflow or equivalent), CI/CD automation, containerized model training and serving, and endpoint registration. |
| Final Submission | 11 December | End-to-end system with deployment, monitoring, and drift management. |

Students are encouraged to develop the project incrementally according to the checkpoints. Adjustments to intermediate deliverables may be made upon request.

The outcome of this project extends beyond the final artifacts themselves, encompassing the demonstrable evidence that the concepts taught throughout the semester were effectively applied, such as consistent code commits, detailed experiment tracking logs, and comprehensive model versioning records.

## 6. Deliverables

Each team must submit a complete record of their work demonstrating both the technical artifacts and the application of MLOps principles taught throughout the semester. The deliverables are divided into required and optional (bonus) components.

### A. Required Deliverables

**GitHub Repository**

Organized, public or shared repository containing all source code, configuration files, and documentation. Demonstrated use of Git for version control (regular commits, branches, and commit messages reflecting the development process).

**Experiment Tracking and Model Versioning**

MLflow or equivalent logs of experiments, including metrics, parameters, and model artifacts. Clear evidence of model registry usage with proper versioning and promotion between stages (e.g., "Staging", "Production").

**CI/CD Pipeline**

Fully functional Continuous Integration, Continuous Delivery, and Continuous Deployment workflow implemented via GitHub Actions (or equivalent). Automatic testing, training, model packaging, and deployment steps documented and reproducible.

**Monitoring**

Evidence of deployed system monitoring (metrics dashboard, logs, or reports) with support for drift detection mechanisms (or similar).

**Documentation**

Technical report describing:
- Dataset and preprocessing steps
- Model architecture and training strategy
- MLOps pipeline design and implementation
- Monitoring and drift detection approach
- Discussion of results and reflections on challenges faced

**Evidence of MLOps Practices**

Proof that the team applied the concepts taught during the semester, such as:
- Meaningful code commits and pull requests
- Experiment tracking logs
- Model versioning history
- Workflow automation scripts

**Self- and Peer-Assessment**

A self-assessment report summarizing how MLOps principles were implemented and lessons learned. An individual peer assessment of team member contributions, evaluating collaboration, effort, and adherence to project goals.

## B. Optional (Bonus) Deliverables

**Continuous Training (CT)**

Automated retraining mechanism triggered by performance degradation or data drift.

**Data Versioning (DVC Integration)**

Implementation of data lineage and version control using DVC, integrated into the CI/CD workflow.

## 7. Evaluation and Grading Matrix

Students must work in **groups of four**.

The checkpoint will take place during a regular class, with each group presenting its progress. All group members must participate, answer questions, and demonstrate their individual contributions. Students who fail to demonstrate meaningful work or contribution will receive a grade of zero. **The checkpoint accounts for 5% of the final grade.**

The final submission accounts for the remaining **95%**. Project defenses will be conducted individually, and every student must demonstrate a thorough understanding of the entire project, including its architecture, implementation, and technical decisions, regardless of who implemented each component. AI tools may support development, but all technical decisions must be justified through sound reasoning and understanding. Referring to an LLM's recommendations/implementations is not an acceptable justification.

Assessment will focus on the adequacy of the model development process and the completeness and quality of the MLOps implementation. Particular emphasis will be placed on **reproducibility, automation, reliability, and engineering quality**, rather than predictive performance alone.

Projects that merely reproduce the provided baseline pipeline without demonstrating substantial independent implementation and application of MLOps concepts may fail to meet the minimum assessment requirements. **Students who fail to demonstrate sufficient understanding of the project and underlying concepts, including work performed by teammates or generated using AI, or who have not made a significant contribution to the project, will receive a grade of zero.**

Continuous Training (CT) and Data Versioning (DVC) are optional components that may contribute to bonus credit.

| Evaluation Dimension | Weight | Description |
|---|---|---|
| Model Training Process (Design & Adequacy) | 20% | Sound feature engineering, appropriate validation design, and clear documentation of the training process. |
| Code Versioning & Reproducibility | 20% | Consistent and correct use of Git and GitHub for version control, collaboration, and documentation. |
| Experiment & Model Versioning (MLflow or | 20% | Accurate logging of parameters, metrics, and artifacts to ensure full traceability of |

| | | |
|---|---|---|
| equivalent) | | experiments. |
| CI/CD Automation | 20% | Reliable automation of testing, training, and deployment workflows (extending beyond the functionality provided in the baseline project). |
| Deployment & Monitoring | 20% | Functional containerized deployment, live inference API, and effective monitoring and drift detection mechanisms. |
| Bonus: Continuous Training & Data Versioning (CT, DVC) | +10% | Optional advanced integration of continuous retraining and/or data versioning pipelines using DVC or automated workflows. |

The final grade will be calculated as the weighted average of the five core evaluation dimensions, with applicable bonus points added. **A minimum grade of 35% is required to pass.**

The highest marks will be awarded to projects demonstrating well-justified modeling decisions, comprehensive automation, rigorous experiment tracking, reproducible workflows, and effective deployment and monitoring. Bonus credit will recognize technically sound extensions that go beyond the mandatory requirements.